SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY



COURSE REPORT: SEMINAR 2

Project title	Multi-label code comment classification on the NLBSE'23 dataset with RoBERTa-large, LoRA and an imbalance-aware training recipe
Group	21
Lecturer	Assoc. Prof. Dr. Nguyen Thanh Phuong
Students	Mai Duc Phuc – 20251239M Ngo Hoang Dang – 20252082M Mai Hoang Long – 20251237M Ninh Van Nghia – 20251043M

Link GitHub: https://github.com/PhucDucMai/Code-Commnet-CLS

Hanoi - 2026

TABLE OF CONTENTS

TABLE OF CONTENTS 1
LIST OF TABLES 2
LIST OF FIGURES 3
LIST OF ABBREVIATIONS 4
INTRODUCTION 5
1. Background and Related Work 6
1.1 Source code comments and their classification *6*
1.2 The NLBSE'23 tool competition and prior solutions *6*
1.3 Multi-label text classification *7*
1.4 Transformer encoders and parameter-efficient fine-tuning *7*
1.5 Learning with imbalanced labels *8*
2. Dataset and Problem Formulation 8
2.1 The NLBSE'23 dataset *8*
2.2 Characteristics and challenges *9*
2.3 Problem formulation *10*
3. Proposed Method 10
3.1 Overview *10*
3.2 Context prompt *11*
3.3 Encoder with Low-Rank Adaptation *12*
3.4 Pooling and classification head *12*
3.5 Asymmetric Loss *12*
3.6 Weighted sampling *13*
3.7 Per-label threshold optimisation *14*
3.8 Training procedure and final model *14*
4. Experimental Setup 14
4.1 Baselines *14*
4.2 Backbones *15*
4.3 Evaluation protocol *16*
4.4 Implementation details *17*
5. Results and Discussion 18
5.1 Overall results *18*
5.2 Comparison with the baselines *19*
5.3 Where the gain comes from *20*
5.4 Comparison of backbones *21*
5.5 Ablation: masked mean pooling versus the first token *22*
5.6 Results per language and per label *23*
5.7 Training dynamics *23*
5.8 Error analysis *24*
6. Demonstration Application 25
7. Limitations and Threats to Validity 27
7.1 Threshold tuning and the evaluation protocol *27*
7.2 Sentence-level folds and the class name in the prompt *27*
7.3 Statistical power and scope of the ablations *27*
CONCLUSION AND FUTURE WORK 28
REFERENCES 29

LIST OF TABLES

Table 1: Dataset statistics by programming language 8
Table 2: The 19 labels, their meaning and their number of sentences 9
Table 3: Loss of a single negative label under binary cross-entropy and Asymmetric Loss 13
Table 4: Controlled comparison: the only differences between the RoBERTa baseline and the full recipe 15
Table 5: Evaluation metrics 16
Table 6: Main hyper-parameters (configs/train_rtx5060.yaml) 17
Table 7: Results of all methods, mean over 5 folds (F1 samples with standard deviation) 18
Table 8: F1 (samples) per fold and paired differences 19
Table 9: Decomposition of the gain: decision rules applied post-hoc to the RoBERTa baseline 20
Table 10: Encoders under the same recipe, compared with RoBERTa-large 21
Table 11: Pooling ablation on RoBERTa-large with the full recipe (mean over 5 folds) 22
Table 12: Macro F1 over the labels of each language 23
Table 13: Most frequent confusions of the main model (true label missed, other label predicted) 24

LIST OF FIGURES

Figure 1: Architecture of the multi-label classifier: context prompt, RoBERTa-large with LoRA, masked mean pooling, MLP head and per-label thresholds 11
Figure 2: Console output of scripts/run_experiments.py: plan, progress of the five runs and final summary 18
Figure 3: F1 (samples) of every method, mean ± standard deviation over the 5 folds 19
Figure 4: F1 (samples) against 5-fold training time; bubble size is the number of parameters 21
Figure 5: Pooling ablation: F1 (samples) of masked mean and first-token pooling on each fold 22
Figure 6: Validation F1 (samples) per epoch of the five transformer runs, and training versus validation loss of the main model 24
Figure 7: Exact match of the main model by subgroup (left) and its most frequent label confusions (right) 24
Figure 8: Web demo on DownloadManager.java: (a) predicted labels and probabilities grouped by class and checked against expected labels; (b) original sentences and model inputs 26

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ASL	Asymmetric Loss
AUC	Area Under the Curve
BCE	Binary Cross-Entropy
BERT	Bidirectional Encoder Representations from Transformers
CLS	Classification token of BERT-style encoders
CV	Cross-Validation
DL	Deep Learning
F1	Harmonic mean of precision and recall
GELU	Gaussian Error Linear Unit
GPU	Graphics Processing Unit
LoRA	Low-Rank Adaptation
LR	Logistic Regression
ML	Machine Learning
MLP	Multi-Layer Perceptron
NLBSE	Natural Language-based Software Engineering (workshop)
NLP	Natural Language Processing
OOF	Out-of-Fold (predictions)
OvR	One-vs-Rest
PEFT	Parameter-Efficient Fine-Tuning
PR-AUC	Area under the Precision-Recall curve
ROC-AUC	Area under the Receiver Operating Characteristic curve
TF-IDF	Term Frequency – Inverse Document Frequency
VRAM	Video Random-Access Memory (GPU memory)

INTRODUCTION

Source code comments explain what software does, how it should be used, and why it was written in a particular way. The *class comment* is usually the first text a developer reads in an unfamiliar class, and it mixes a summary, usage instructions, pointers to related elements, deprecation warnings, authorship notes and design rationale. Recognising these kinds of information automatically supports documentation quality assessment, code search and documentation tools.

The NLBSE'23 tool competition on code comment classification [1], [2] formalised this task: every sentence of a class comment, taken from Java, Python and Pharo projects, has to be assigned the categories of a language-specific taxonomy. A sentence can belong to several categories at once, which makes the problem a *multi-label* text classification task. The sentences are short, the categories are strongly imbalanced, and several categories have overlapping meanings, so a model that simply predicts the most frequent label for each sentence performs poorly.

This report presents the work of Group 21 for the course Seminar 2. The assignment asked us to build machine learning (ML) and deep learning (DL) classifiers for multi-label code comment classification, to choose evaluation metrics suited to the task, to evaluate the models with k-fold cross-validation, and to compare them with a dedicated Transformer baseline based on RoBERTa. Our objectives were therefore:

- to build a single multi-label classifier for all 19 categories on a RoBERTa-large encoder adapted with LoRA, together with a training recipe designed for imbalanced multi-label data;
- to compare it fairly with a classical ML baseline (TF-IDF features with logistic regression) and a plainly fine-tuned RoBERTa baseline, on exactly the same five cross-validation folds;
- to measure where the improvement comes from, whether it holds across other encoders, and whether design choices such as the pooling strategy matter;
- to analyse the remaining errors and the validity of the evaluation, and to turn the trained model into a usable demonstration tool.

With the same backbone and compute budget, the training recipe raises the sample-averaged F1 score from 0.5721 for plain fine-tuning to **0.8124 ± 0.0064**, consistently on all five folds (paired t-test, $p = 1.4 \times 10^{-5}$). Chapter 1 reviews related work, Chapter 2 describes the data, Chapter 3 the method and Chapter 4 the experimental setup. Chapter 5 reports the results, Chapter 6 the demonstration tool and Chapter 7 the limitations; the last chapter concludes.

1. Background and Related Work

1.1 Source code comments and their classification

Code comments complement identifiers and types with natural-language explanations, and their quality directly affects program comprehension and maintenance. Rani et al. [1] studied class comments in Java, Python and Smalltalk (Pharo) and showed that, although each language community has its own conventions, class comments repeatedly contain the same kinds of information, and they proposed a taxonomy of comment types for each language together with a manually labelled dataset of class comment sentences.

1.2 The NLBSE'23 tool competition and prior solutions

The NLBSE'23 tool competition [2], organised with the International Workshop on Natural Language-based Software Engineering, released the dataset of Rani et al. in sentence form with a fixed training and test split. The official protocol treats each of the 19 language-specific categories as a separate binary classification problem and reports precision, recall, and F1 for every category. The organisers provided a baseline based on Random Forest classifiers with TF-IDF and NLP features, one classifier per category.

STACC [3], one of the competition submissions, fine-tuned 19 binary sentence-transformer classifiers with the SetFit framework [4], which combines contrastive fine-tuning of a sentence embedding model with a light classification head. STACC improved clearly over the Random Forest baseline, but because it trains one model per category, 19 models have to be trained and run, and rare categories cannot share information with frequent ones.

1.3 Multi-label text classification

In multi-label classification, each instance is associated with a subset of a label set rather than with exactly one label [5]. The simplest approach, *binary relevance*, trains one independent binary classifier per label; this is what the official competition protocol and STACC do. Neural networks offer a more compact alternative: a single shared encoder with one sigmoid output per label, trained with a per-label binary loss. The shared representation lets related labels benefit from each other, and a single forward pass produces all predictions.

Evaluation is also different from single-label classification. Plain accuracy is not meaningful, because a prediction can be partially correct. Common metrics include *sample-averaged F1* (the F1 of each instance's predicted label set, averaged over instances), micro- and macro-averaged F1 over labels, the Jaccard index, exact match (subset accuracy) and the Hamming loss, as well as threshold-independent ranking metrics such as ROC-AUC and the area under the precision-recall curve (PR-AUC). Cross-validation folds should be built with *iterative stratification* [6], which preserves the frequency of every label in every fold.

1.4 Transformer encoders and parameter-efficient fine-tuning

BERT [7] established pretraining a Transformer encoder and fine-tuning it on a downstream task; RoBERTa [8] trains longer on more data and removes the next-sentence prediction objective. DeBERTa-v3 [9] adds disentangled attention and ELECTRA-style replaced-token detection; ModernBERT [10] is a recent encoder trained on a large corpus that includes code, with architectural updates for efficiency; UniXcoder [11] is pretrained on source code and its comments with objectives designed for code representation. These four encoders are the backbones compared in this work.

Fully fine-tuning a large encoder updates hundreds of millions of parameters and requires storing gradients and optimiser states for all of them. Low-Rank Adaptation (LoRA) [12] freezes the pretrained weights and learns, for selected weight matrices W, a low-rank update $B \cdot A$ with rank r much smaller than the matrix dimensions. Only A, B, and the task head are trained, which reduces memory use and checkpoint size by an

order of magnitude while keeping accuracy close to full fine-tuning. We use LoRA through the PEFT library [13] on top of Hugging Face Transformers [14].

1.5 Learning with imbalanced labels

In multi-label learning, every instance has a few positive labels and many negative ones, and with binary cross-entropy (BCE), the many easy negatives dominate the gradient. Three families of remedies exist. *Loss re-weighting* changes the contribution of each example: Focal Loss [15] down-weights easy examples, and Asymmetric Loss (ASL) [16] treats positives and negatives differently, strongly down-weighting easy negatives and discarding very easy ones. *Re-sampling* changes how often each example is seen, for instance by drawing examples with rare labels more often. *Decision-rule adjustment* replaces the default 0.5 threshold with thresholds tuned per label on validation data. Our training recipe combines one method from each family.

2. Dataset and Problem Formulation

2.1 The NLBSE'23 dataset

We use the sentence-level dataset of the NLBSE'23 competition, stored as data/raw/sentences.csv. Each row contains a sentence identifier, the identifier of the class it comes from, the sentence text, the original partition (training or test), the semicolon-separated labels, and the programming language. The dataset has 6,738 sentences from 1,006 classes, about 6.7 sentences per class. Table 1 summarises it by language.

Table 1: Dataset statistics by programming language

Language	Sentences	Labels	Label names
Java	2,418	7	summary, usage, expand, pointer, deprecation, rationale, ownership
Python	2,555	5	summary, usage, expand, parameters, developmentnotes
Pharo	1,765	7	intent, responsibilities, collaborators, keymessages, example, keyimplementationpoints, classreferences
Total	**6,738**	**19**	**16 distinct names; summary, usage and expand are split by language**

The taxonomy contains 16 distinct label names. The categories summary, usage, and expand exist in both Java and Python but are annotated with language-specific conventions, so we split them by language (for example, summary_java and summary_python). This gives the 19 labels used throughout the report and listed in Table 2 with their meaning and frequency.

Table 2: The 19 labels, their meaning, and their number of sentences

Label	Language	Meaning	Sentences
summary_java	Java	What the class is or does, in one or two sentences	415
usage_java	Java	How to use the class: calls, expected inputs, examples	912
expand_java	Java	Further details on behaviour or implementation	632
pointer	Java	References to other classes, methods or resources (@see, @link, URLs)	364
deprecation	Java	The class or API is deprecated, what replaces it	127
rational	Java	Why a design or implementation choice was made	280
ownership	Java	Authors, owners, versions (@author, @since, @version)	115
summary_python	Python	What the class is or does, in one or two sentences	454
usage_python	Python	How to use the class: calls, expected inputs, examples	800
expand_python	Python	Further details on behaviour or implementation	504
parameters	Python	Arguments and attributes and their meaning (Args:, Attributes:)	794
developmentnotes	Python	Notes for developers: TODOs, known limitations, future work	312
intent	Pharo	The purpose of the class, usually "I am / I represent ..."	218
responsibilities	Pharo	What the class knows and does	336
collaborators	Pharo	Other classes it works with and how	127
keymessages	Pharo	The main messages (methods) of its public API	305
example	Pharo	Example code showing how to use the class	748
keyimplementationpoints	Pharo	Internal representation and implementation details	232
classreferences	Pharo	Mentions of other related classes	77

2.2 Characteristics and challenges

Three properties make the task difficult. Labels are **imbalanced**: usage_java has 912 sentences and classreferences only 77, about twelve times fewer. Inputs are **very short**: the median sentence has six words, and 751 lines are a single word, often an HTML tag left over from Javadoc. The task is **multi-label**: 912 sentences (13.5%)

carry two to four labels, and categories such as expand, summary, and usage overlap. On the other hand, sentences are not independent: 80% of adjacent sentences in a class share identical labels, so the language and the class are useful context. The original split (5,392 training and 1,346 test sentences) is not used by our cross-validation (see Chapter 7).

The sentences are already pre-processed by the dataset authors: they are lower-cased, most punctuation is removed, and a dot inside an identifier is replaced by the character ^. For example, the Javadoc fragment {@link Foo#bar()} appears as @link foo#bar. Our demonstration tool (Chapter 6) reproduces exactly this normalisation for new source files.

2.3 Problem formulation

Let L = {1, …, 19} be the label set. Each example i consists of a sentence s_i, its language $g_i \in$ {Java, Python, Pharo} and its class identifier c_i, and is annotated with a label vector $y_i \in \{0, 1\}^{19}$, where $y_{i,j} = 1$ if label j applies. A model f produces a probability for every label, and a vector of thresholds t turns these probabilities into a predicted label set:

$$p_i = f(s_i, g_i, c_i) \in [0, 1]^{19}, \qquad \hat{y}_{i,j} = \mathbb{1}[\, p_{i,j} \geq t_j \,] \tag{1}$$

Our main evaluation metric is the sample-averaged F1 score, which compares the predicted and true label sets of each sentence and averages over the N sentences:

$$F1_samples = (1/N) \sum_i 2\,|\hat{Y}_i \cap Y_i| \,/\, (\,|\hat{Y}_i| + |Y_i|\,) \tag{2}$$

A sentence with no predicted label scores zero, as does a sentence whose predicted labels are all wrong. We chose this metric because it matches how the classifier is used, sentence by sentence, and because it rewards getting every label of a multi-label sentence right. We also report seven complementary metrics, defined in Section 4.3.

3. Proposed Method

3.1 Overview

Figure 1 shows the complete model. A sentence is first combined with its language and class name into a short *context prompt*. The prompt is tokenised and

encoded by RoBERTa-large adapted with LoRA. The token representations are averaged into a single sentence vector, which a small multi-layer perceptron (MLP) maps to 19 logits. A sigmoid turns each logit into a probability, and a separate threshold per label produces the final label set. During training, two further components act on the data and the loss: a weighted sampler and the Asymmetric Loss.

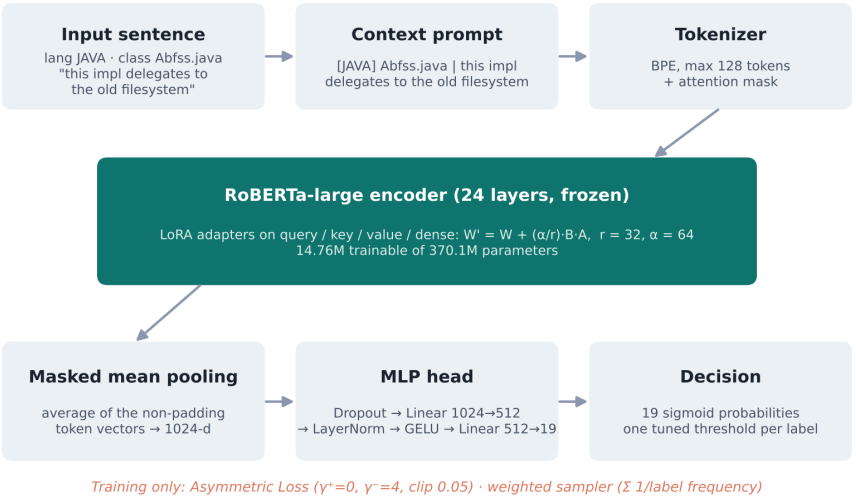


Figure 1: Architecture of the multi-label classifier: context prompt, RoBERTa-large with LoRA, masked mean pooling, MLP head and per-label thresholds

We use **one multi-label model** instead of 19 binary models. This has three practical advantages: a single forward pass produces all labels, rare labels share an encoder with frequent ones, and only one checkpoint and one threshold vector have to be deployed. The training recipe that differs from plain fine-tuning consists of four components, described in Sections 3.2, 3.5, 3.6 and 3.7. These components were first configured in the project's original code base [20]; in this work we make the recipe run on a consumer GPU, compare it against controlled baselines, and analyse it in depth.

3.2 Context prompt

Because sentences are short and neighbouring sentences of a class tend to share labels, we prepend the language and the class identifier to every sentence using the template `[{LANG}] {class_id} | {sentence}`. For example, the Java sentence "this impl delegates to the old filesystem" from class Abfss.java becomes:

[JAVA] Abfss.java | this impl delegates to the old filesystem

The language tag tells the model which part of the taxonomy applies (for instance, parameters exist only in Python and intent only in Pharo), and the class name gives a hint about the domain of the class. Inputs are truncated to 128 tokens, which is never reached in practice because the median prompt is far shorter. Section 7.2 discusses a side effect of including the class name under a sentence-level evaluation.

3.3 Encoder with Low-Rank Adaptation

The encoder is RoBERTa-large [8], a 24-layer Transformer with hidden size 1024. Its pretrained weights are frozen. For every linear layer named query, key, value, and dense in all 24 layers, LoRA [12] adds a trainable low-rank update:

$$W' = W + (\alpha / r) \cdot B\,A, \qquad B \in \mathbb{R}^{(d \times r)},\ A \in \mathbb{R}^{(r \times k)},\ r = 32,\ \alpha = 64 \tag{3}$$

With LoRA dropout 0.05 and the classification head, 14.76 million of the 370.1 million parameters are trained (3.99%). This keeps the peak GPU memory at 4.8 GB with bfloat16 mixed precision, so the model trains on an 8 GB consumer GPU, and it shrinks each checkpoint from about 1.4 GB to about 57 MB because only the trainable parameters are saved. For the other backbones in Section 4.2, the LoRA target layers are chosen automatically according to the architecture.

3.4 Pooling and classification head

The encoder outputs one vector $h_t \in \mathbb{R}^{1024}$ per token. Since the input is padded to a fixed length and real sentences are short, most positions are padding. We therefore use **masked mean pooling**, which averages only the vectors of real tokens, using the attention mask $m_t \in \{0, 1\}$:

$$h = \Sigma_t\, m_t\, h_t \,/\, \Sigma_t\, m_t \tag{4}$$

We prefer it to the first-token vector (<s>, the equivalent of [CLS]) because RoBERTa's first token is not pretrained as a sentence summary and mean pooling gives robust sentence representations [17]; Section 5.5 tests this choice. The pooled vector is passed to a two-layer head:

$$z = W_2 \cdot GELU(\, LayerNorm(\, W_1 \cdot Dropout(h)\,)\,) \tag{5}$$

where $W_1 \in \mathbb{R}^{(512 \times 1024)}$ and $W_2 \in \mathbb{R}^{(19 \times 512)}$, with dropout before each linear layer; the probabilities are $p_i = \sigma(z_i)$.

3.5 Asymmetric Loss

Each sentence has on average 1.15 positive labels and about 18 negative ones. With BCE, the loss of the negatives dominates, and the model learns to predict low probabilities for everything. We use the Asymmetric Loss [16] with $\gamma^+ = 0$, $\gamma^- = 4$, and a probability margin m = 0.05. For a label with predicted probability p, the positive and negative terms are:

$$L^+ = -(1 - p)^{\gamma^+} \cdot \log(p) = -\log(p) \quad (6)$$

$$L^- = -p^{\gamma^-} \cdot \log(\min(1 - p + m, 1)) \quad (7)$$

The positive term is ordinary cross-entropy. The negative term is multiplied by p^4, which almost removes the contribution of easy negatives, and the margin sets the loss to exactly zero when $p < 0.05$. The final loss is the mean over all labels of the batch. Table 3 makes the effect concrete for a single negative label.

Table 3: Loss of a single negative label under binary cross-entropy and Asymmetric Loss

Predicted p	Interpretation	BCE $-\log(1-p)$	ASL L^-	Share of BCE kept
0.03	very easy negative	0.030	0	0%
0.10	easy negative	0.105	0.000005	≈ 0%
0.50	uncertain	0.693	0.037	5%
0.90	hard error	2.303	1.245	54%

Hard errors are still penalised, but the many negatives that are already classified correctly no longer push the probabilities of the true labels down.

3.6 Weighted sampling

To show rare labels more often during training, each training sentence i is drawn with a probability proportional to the sum of the inverse frequencies of its labels in the training fold:

$$w_i = \Sigma_j y_{ij} / (n_j + \varepsilon), \quad n_j = \text{number of training sentences with label } j, \quad \varepsilon = 10^{-3} \quad (8)$$

Sentences without any label receive half of the smallest label weight. Sampling is done with replacement, and each epoch draws as many sentences as the training fold

contains. A sentence labelled classreferences (about 62 training sentences per fold) is drawn roughly twelve times more often than one labelled usage_java (about 730).

3.7 Per-label threshold optimisation

A fixed threshold of 0.5 is a poor decision rule under imbalance: the probabilities of rare labels are often below 0.5 even when they are ranked first. After each epoch, we therefore tune one threshold per label on the validation fold, in two steps:

1. For each label j independently, scan 100 values of t□ between 0.1 and 0.9 and keep the value that maximises the binary F1 score of label j.
2. Starting from these values, run two rounds of coordinate descent: for each label in turn, scan 50 values between 0.05 and 0.95 while keeping the other thresholds fixed, and keep the value that maximises F1 (samples) over all labels.

The second step aligns the thresholds with the main metric, which depends on all labels of a sentence jointly. Across the five folds, the median thresholds of the main model range from 0.46 for ownership, a very reliable label, to 0.80 for pointer, which tends to produce false positives. The median over the folds is stored with the model and used at inference time.

3.8 Training procedure and final model

Each fold is trained with AdamW [18] for at most 30 epochs (hyperparameters in Table 6). After every epoch, the model is evaluated on the validation fold with tuned thresholds, and the epoch with the best F1 (samples) is kept.

After cross-validation, a **final model** for the demonstration is trained on all 6,738 sentences. It is trained for 25 epochs, the rounded mean best epoch of cross-validation (25.4), on the same 30-epoch learning-rate schedule, so that it ends at the same point of the schedule as the fold models and the median fold thresholds remain appropriate. The final model took 30.4 minutes to train.

4. Experimental Setup

4.1 Baselines

The assignment requires a comparison with a classical ML approach and with a Transformer baseline based on RoBERTa. Both baselines use exactly the same folds, threshold procedure (where applicable), and metrics as our model, and write their results in the same format.

TF-IDF + Logistic Regression (ML baseline)

The sentence is represented by the concatenation of word TF-IDF features (unigrams and bigrams) and character TF-IDF features (character 2- to 5-grams within word boundaries, minimum document frequency 2), both with sub-linear term frequency, about 49,000 features in total. The vectorisers are fitted on each training fold only. A one-vs-rest logistic regression (C = 4, balanced class weights, liblinear solver) is trained for each label, and per-label thresholds are tuned with the same procedure as in Section 3.7. The baseline runs on a CPU in about 30 seconds for all five folds. It is implemented in scripts/ml_baseline.py, which also supports a Random Forest variant, the kind of baseline used by the competition organisers.

Plain RoBERTa fine-tuning (Transformer baseline)

To measure the effect of the training recipe, the Transformer baseline uses the **same** model as our method: RoBERTa-large with the same LoRA configuration, the same pooling and head, the same optimiser, learning rate, batch size, number of epochs, early stopping and folds. Only the four recipe components are switched off, as shown in Table 4. Any difference between the two runs can therefore be attributed to these four components.

Table 4: Controlled comparison: the only differences between the RoBERTa baseline and the full recipe

Component	RoBERTa baseline	Full recipe	Problem addressed
Input	sentence only	prompt [LANG] class_id | sentence	short sentences
Loss	binary cross-entropy	Asymmetric Loss ($\gamma^+ = 0$, $\gamma^- = 4$, $m = 0.05$)	many easy negatives
Sampling	uniform random	weighted: Σ 1 / label frequency	rare labels

Component	RoBERTa baseline	Full recipe	Problem addressed
Decision	threshold 0.5 for all labels	one tuned threshold per label	multi-label decisions

4.2 Backbones

To test whether the results depend on RoBERTa, we apply the identical recipe and hyperparameters to three other encoders: DeBERTa-v3-large [9] and ModernBERT-large [10], pretrained on general text (ModernBERT also on code), and UniXcoder-base [11], pretrained on source code and comments. Only the LoRA target layers change, because their names differ between architectures; their sizes are given in Table 10.

4.3 Evaluation protocol

Cross-validation

All methods are evaluated with 5-fold cross-validation on the 6,738 sentences. The folds are built with MultilabelStratifiedKFold (iterative stratification [6]) with seed 42, so every fold preserves the frequency of all 19 labels and every method sees exactly the same training and validation sentences. For each fold, the model of the best epoch and its tuned thresholds are kept, and its predictions on the validation fold are saved. Every sentence is therefore predicted exactly once by a model that did not see it during training; we call these *out-of-fold* (OOF) predictions. They allow paired statistical tests, ensembles and error analysis without retraining.

Metrics

Table 5 lists the eight metrics we report. F1 (samples) is the main metric; it is also used for model selection and threshold tuning. The threshold-independent PR-AUC and ROC-AUC measure the quality of the ranking of labels and are not affected by the decision rule.

Table 5: Evaluation metrics

Metric	What it measures	Better
F1 (samples)	F1 of each sentence's label set, averaged over sentences (main metric)	higher
F1 (micro)	F1 over all label decisions pooled together	higher
F1 (macro)	Mean of the 19 per-label F1 scores; every label counts equally	higher

Metric	What it measures	Better
PR-AUC (macro)	Area under the precision-recall curve, averaged over labels; no threshold	higher
ROC-AUC (macro)	Area under the ROC curve, averaged over labels; no threshold	higher
Jaccard (samples)	Overlap $\lvert\hat{Y} \cap Y\rvert / \lvert\hat{Y} \cup Y\rvert$ of predicted and true label sets	higher
Exact match	Share of sentences whose whole label set is predicted correctly	higher
Hamming loss	Share of wrong label decisions over all sentences and labels	lower

Statistical tests

Because all methods share the folds, we compare two methods with a **paired t-test** on their five per-fold F1 (samples) scores and report how many folds each wins. For the pooling ablation, where differences are small, we add two sentence-level tests on the OOF predictions: a **bootstrap** confidence interval of the difference in F1 (samples) with 10,000 resamples of the 6,738 sentences [19], and **McNemar's test** [21] on exact match, which only uses the sentences on which the two models disagree.

4.4 Implementation details

The code is written in Python with PyTorch, Hugging Face Transformers [14], PEFT [13], scikit-learn [22], and iterative-stratification. src/train.py implements the full pipeline: data loading, prompt construction, model, losses, sampler, threshold search, metrics, logging, checkpoints, and the final model. Table 6 lists the main hyperparameters of the main configuration, configs/train_rtx5060.yaml.

Table 6: Main hyper-parameters (configs/train_rtx5060.yaml)

Group	Setting
Model	roberta-large, max length 128, masked mean pooling, MLP 1024 → 512 → 19
LoRA	r = 32, α = 64, dropout 0.05, on query, key, value, dense
Optimisation	AdamW, lr 5×10^{-5}, weight decay 0.01, cosine schedule, 10% warm-up
Batch	16 × 4 gradient accumulation = 64; bfloat16 mixed precision
Epochs	≤ 30; early stopping patience 5; never before epoch 10
Loss and sampling	ASL $\gamma^{+} = 0$, $\gamma^{-} = 4$, clip 0.05; weighted sampler with replacement
Decision	per-label thresholds: 100-point grid, then 2 rounds of coordinate descent on F1 (samples)
Cross-validation	MultilabelStratifiedKFold, 5 folds, seed 42

Group	Setting
Final model	all 6,738 sentences, 25 epochs on the 30-epoch schedule, median fold thresholds

All experiments ran on a single NVIDIA RTX 5060 GPU with 8 GB of memory. The script scripts/run_experiments.py trains every model listed in configs/experiments.yaml with one command: finished runs are skipped, interrupted runs resume from their last completed fold, and a run that exhausts GPU memory is retried with half the batch size and twice the gradient accumulation, which keeps the effective batch size unchanged. The overnight run that produced the main model's final checkpoint, the RoBERTa baseline, and the three other backbones took 11 hours 34 minutes (Figure 2); during training, every epoch prints a report with the loss, learning rate, all metrics, and per-language F1.

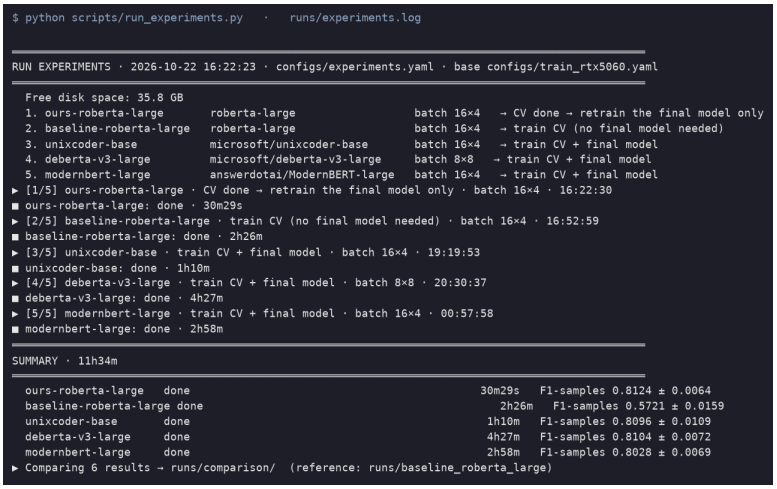


Figure 2: Console output of scripts/run_experiments.py: plan, progress of the five runs and final summary

5. Results and Discussion

5.1 Overall results

Table 7 reports the mean of the eight metrics over the five folds for every method, and Figure 3 shows F1 (samples) with its standard deviation. The full recipe on RoBERTa-large reaches **0.8124 ± 0.0064** F1 (samples), against 0.6015 for TF-IDF + LR and 0.5721 for plain RoBERTa fine-tuning. It is better than both baselines on every metric.

Table 7: Results of all methods, mean over 5 folds (F1 samples with standard

deviation)

Method	F1 samples	F1 micro	F1 macro	PR-AUC	ROC-AUC	Jaccard	Exact	Hamming ↓	CV time
TF-IDF + LR	0.6015 ± 0.0127	0.5871	0.5893	0.6194	0.9155	0.5470	0.3973	0.0594	30 s (CPU)
RoBERTa baseline	0.5721 ± 0.0159	0.6050	0.5743	0.6194	0.9177	0.5571	0.5129	0.0432	2 h 26 min
Full recipe · RoBERTa-large	**0.8124 ± 0.0064**	**0.7973**	**0.7829**	0.8198	0.9665	**0.7790**	**0.6805**	**0.0260**	2 h 41 min
Full recipe · UniXcoder-base	0.8096 ± 0.0109	0.7956	0.7821	**0.8233**	**0.9712**	0.7733	0.6658	0.0264	1 h 01 min
Full recipe · DeBERTa-v3-large	0.8104 ± 0.0072	0.7910	0.7743	0.8088	0.9556	0.7753	0.6726	0.0270	3 h 46 min
Full recipe · ModernBERT-large	0.8028 ± 0.0069	0.7851	0.7653	0.8025	0.9599	0.7684	0.6679	0.0277	2 h 30 min
Ensemble of the 4 (post-hoc)	0.8382 ± 0.0076	0.8214	0.8087	0.8614	0.9815	0.8049	0.7063	0.0230	—

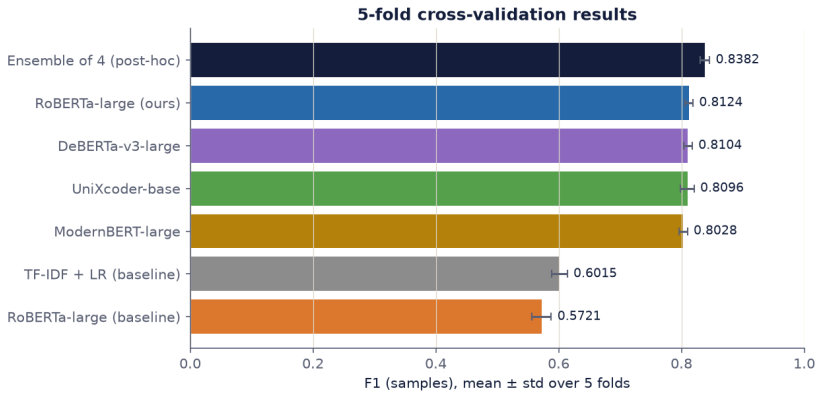


Figure 3: F1 (samples) of every method, mean ± standard deviation over the 5 folds

The gain is not limited to the main metric. Exact match rises from 0.5129 to 0.6805, so two out of three sentences now receive exactly the right label set, and the Hamming loss drops from 0.0432 to 0.0260. The threshold-independent PR-AUC rises from 0.6194 to 0.8198, which shows that the recipe improves the *ranking* of the labels and not only the decision rule. The best single model is RoBERTa-large on most metrics, while UniXcoder-base has the best PR-AUC and ROC-AUC.

5.2 Comparison with the baselines

Table 8 shows F1 (samples) on each fold. The full recipe is better than both baselines on all five folds. The improvement over the RoBERTa baseline ranges from +0.206 to +0.262, with a mean of **+0.2403** ($t = 25.39$, $p = 1.4 \times 10^{-5}$); the improvement over TF-IDF + LR has a mean of **+0.2110** ($t = 28.84$, $p = 8.6 \times 10^{-6}$).

Table 8: F1 (samples) per fold and paired differences

Method	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean
TF-IDF + LR	0.5893	0.6241	0.5901	0.6047	0.5990	0.6015
RoBERTa baseline	0.5655	0.6019	0.5543	0.5703	0.5685	0.5721
Full recipe (RoBERTa-large)	0.8071	0.8080	0.8161	0.8233	0.8076	0.8124
Δ vs RoBERTa baseline	**+0.2415**	**+0.2062**	**+0.2617**	**+0.2531**	**+0.2390**	**+0.2403**
Δ vs TF-IDF + LR	**+0.2177**	**+0.1839**	**+0.2260**	**+0.2186**	**+0.2085**	**+0.2110**

A surprising observation is that the plainly fine-tuned RoBERTa baseline is *below* the TF-IDF baseline on F1 (samples), although it is better on F1 (micro), exact match, and Hamming loss. The OOF predictions explain why: with BCE and a fixed 0.5 threshold, the RoBERTa baseline predicts no label at all for 13.6% of the sentences, each of which scores zero, and predicts only 0.93 labels per sentence while sentences have 1.15 on average. TF-IDF + LR, whose thresholds are tuned, leaves 4.9% of the sentences empty but strongly over-predicts (1.58 labels per sentence). The full recipe leaves only 1.0% of the sentences empty and predicts 1.28 labels per sentence, much closer to the true count.

5.3 Where the gain comes from

The RoBERTa baseline differs from the full recipe in four components. Separating the effect of each one would require many additional training runs, but the decision rule can be studied without retraining, by applying different rules to the baseline's saved OOF probabilities. Table 9 shows the result.

Table 9: Decomposition of the gain: decision rules applied post-hoc to the RoBERTa baseline

Step	F1 (samples)	Change
RoBERTa baseline, threshold 0.5	0.5721	—
+ at least one label per sentence	0.6043	+0.032
+ per-label thresholds (re-tuned per fold)	0.6317	+0.027
+ thresholds and at least one label	0.6320	+0.000
Full recipe (+ context prompt, ASL, weighted sampler)	**0.8124**	**+0.180**

Better decision rules alone raise the baseline from 0.572 to about 0.632, so they explain roughly **+0.06** of the gain. With the same threshold tuning, the RoBERTa baseline (0.632) is already above TF-IDF + LR (0.602), which confirms that its low score in Table 7 is mostly caused by the fixed threshold. The remaining **+0.18** comes from the three training-time components, the context prompt, the Asymmetric Loss, and the weighted sampler, measured together. The rise of the threshold-independent PR-AUC from 0.62 to 0.82 is consistent with this: the training changes improve the model's scores themselves, not only how they are thresholded.

5.4 Comparison of backbones

Table 10 compares the four encoders trained with the identical recipe, and Figure 4 relates their accuracy to their training cost.

Table 10: Encoders under the same recipe, compared with RoBERTa-large

Encoder	F1 samples	Δ vs RoBERTa	RoBERTa wins	p (t-test)	Params	Peak VRAM	CV time
RoBERTa-large	0.8124	—	—	—	370.1M	4.8 GB	2 h 41 min
DeBERTa-v3-large	0.8104	−0.0020	4 / 5	0.40	448.7M	5.4 GB	3 h 46 min
UniXcoder-base	0.8096	−0.0028	3 / 5	0.50	131.6M	1.8 GB	1 h 01 min
ModernBERT-large	0.8028	−0.0096	5 / 5	0.03	409.7M	5.1 GB	2 h 30 min
Ensemble of the 4	0.8382	+0.0258	0 / 5	1.5×10^{-4}	—	—	—

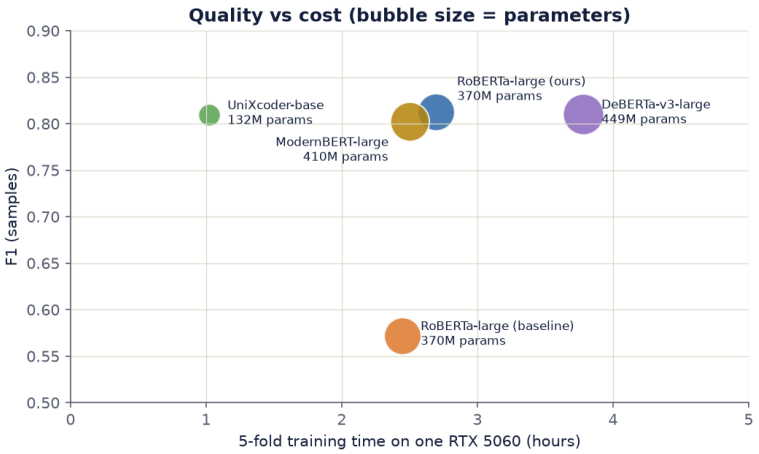


Figure 4: F1 (samples) against 5-fold training time; bubble size is the number of parameters

With the same recipe, the four encoders end **within 0.01** of each other. DeBERTa-v3-large and UniXcoder-base are statistically

indistinguishable from RoBERTa-large ($p \geq 0.40$). ModernBERT-large is lower on all five folds, but a p-value of 0.03 on five folds is weak evidence. The cost differs much more than the accuracy: UniXcoder-base, which is pretrained on source code and its comments, matches RoBERTa-large with 2.8 times fewer parameters, 2.6 times less training time and 2.7 times less GPU memory, which makes it the most cost-effective choice. Together with Section 5.3, this supports the main conclusion of the report: **the training recipe matters far more than the choice of encoder**.

Averaging the OOF probabilities of the four models and re-tuning the thresholds per fold gives an ensemble with 0.8382 F1 (samples), better than RoBERTa-large on all five folds (mean +0.0258, $p = 1.5 \times 10^{-4}$), with the best value of every metric in Table 7, at four times the inference cost of a single model.

5.5 Ablation: masked mean pooling versus the first token

Our model represents a sentence by the masked mean of its token vectors (Section 3.4). To test it against the usual first-token vector (<s>, RoBERTa's [CLS]), we trained the main configuration again with first-token pooling (configs/ablation_cls_pooling.yaml); the pooling is the only change, and the run uses the same folds and seed. Table 11 and Figure 5 compare the two runs.

Table 11: Pooling ablation on RoBERTa-large with the full recipe (mean over 5 folds)

Sentence vector	F1 samples	F1 micro	F1 macro	PR-AUC	Exact match	Hamming ↓	CV time
Masked mean (main model)	0.8124 ± 0.0064	0.7973	0.7829	0.8198	0.6805	0.0260	2 h 41 min
First token <s>	0.8089 ± 0.0074	0.7950	0.7803	0.8176	0.6751	0.0262	2 h 50 min

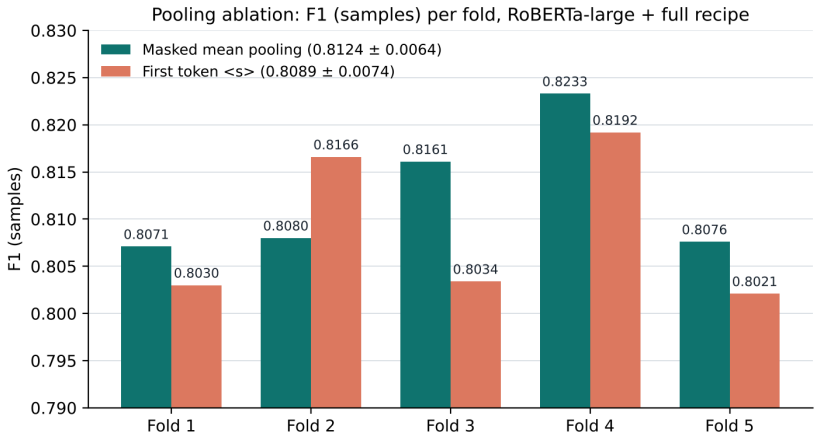


Figure 5: Pooling ablation: F1 (samples) of masked mean and first-token pooling on each fold

Mean pooling is slightly better on average, by **+0.0035 F1 (samples)**, and on four of the five folds, but the difference is smaller than the fold-to-fold standard deviation (about 0.007) and is not statistically significant. (paired t-test $t = 1.03$, $p = 0.36$). On the sentence level, the bootstrap 95% confidence interval of the difference, about [−0.002, +0.009], contains zero, and for exact match, 496 sentences are right only with mean pooling and 460 only with the first token (McNemar, $p = 0.26$).

The effect is also mixed across languages and labels: mean pooling is better on Python (+0.0078 macro F1) and Pharo (+0.0071) but worse on Java (−0.0057), and the per-label differences go in both directions. We conclude that **the pooling strategy is not where the performance comes from**: once the encoder is fine-tuned, both the first token and the mean of the tokens can carry the information the head needs. We keep masked mean pooling because it is marginally better, applies identically to every encoder and makes no assumption about how a particular model was pretrained.

5.6 Results per language and per label

Table 12 reports the macro F1 over the labels of each language. Java is the easiest language for every method, and Python the hardest for the transformer models. The

recipe improves every language by a large margin, about +0.16 for Java, +0.26 for Python and +0.22 for Pharo over the RoBERTa baseline.

Table 12: Macro F1 over the labels of each language

Method	Java (7 labels)	Python (5 labels)	Pharo (7 labels)
TF-IDF + LR	0.6861	0.5197	0.5423
RoBERTa baseline	0.6837	0.4733	0.5372
Full recipe · RoBERTa-large	**0.8478**	0.7341	**0.7529**
Full recipe · UniXcoder-base	0.8463	**0.7357**	0.7510
Full recipe · DeBERTa-v3-large	**0.8560**	0.7219	0.7301
Full recipe · ModernBERT-large	0.8459	0.7213	0.7162

Per label, every one of the 19 labels improves over the RoBERTa baseline. The largest gains are on the hardest labels: developmentnotes rises from 0.08 to 0.58, keyimplementationpoints from 0.40 to 0.77 and summary_python from 0.46 to 0.76, which is the behaviour expected from a recipe designed for imbalance. Labels with a clear surface form are easy for all methods: ownership (@author, @version) is recognised almost perfectly. The weakest labels are collaborators, developmentnotes and classreferences, which are rare, semantically vague, or both.

5.7 Training dynamics

Figure 6 shows the validation F1 per epoch of the five transformer runs and the training and validation loss of the main model. The main model reaches about 0.78 F1 (samples) at epoch 10, and its best epochs are 23 to 28. Its validation loss is lowest around epochs 12 to 17 and then rises slightly while F1 keeps improving: the probabilities become more confident while the ranking of labels still improves, and the per-label thresholds compensate for the calibration. The RoBERTa baseline predicts no label at all during its first three epochs, a direct symptom of the imbalance problem discussed in Section 3.5.

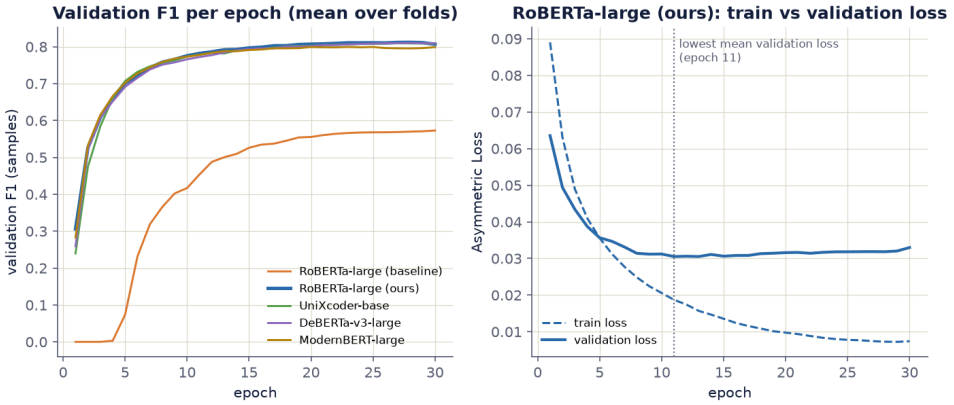


Figure 6: Validation F1 (samples) per epoch of the five transformer runs, and training versus validation loss of the main model

5.8 Error analysis

We analysed the OOF predictions of the main model to understand its remaining errors (Figure 7). Most errors are confusions between neighbouring categories of the same language, listed in Table 13: a true label is missed while another label of the same language is predicted instead.

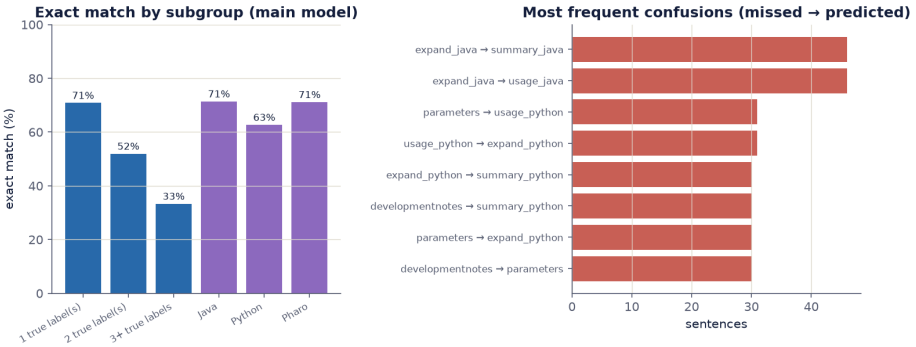


Figure 7: Exact match of the main model by subgroup (left) and its most frequent label confusions (right)

Table 13: Most frequent confusions of the main model (true label missed, other label predicted)

Missed label	Predicted instead	Sentences
expand_java	summary_java	46
expand_java	usage_java	46
parameters	usage_python	31
usage_python	expand_python	31
expand_python	summary_python	30
developmentnotes	summary_python	30
parameters	expand_python	30

Missed label	Predicted instead	Sentences
developmentnotes	parameters	30

Three patterns stand out. First, **overlapping meanings**: expand, summary, and usage are the most confused labels in both Java and Python, and Python's parameters and developmentnotes are confused with the generic categories. These categories genuinely overlap; a sentence that describes behaviour in detail can reasonably be read as either a summary or an expansion. Second, **more labels are harder**: exact match is 70.9% for sentences with one label (n = 5,826), 52.0% for two labels (n = 816) and 33.3% for three or four (n = 96). Third, **slight over-prediction**: the model predicts 1.28 labels per sentence for 1.15 true labels, and its recall (0.857) is higher than its precision (0.800), because thresholds that maximise F1 (samples) favour recall. By language, exact match is 71.4% for Java, 71.2% for Pharo and 62.7% for Python, consistent with the per-language results.

Compared with the RoBERTa baseline, the full recipe gives a better F1 on 2,360 sentences (35.0%), the same F1 on 3,849 (57.1%), and a worse F1 on 529 (7.9%). The worse cases are mostly sentences where an extra label is predicted, the price of the recall-oriented thresholds.

6. Demonstration Application

To show the model in use, we built a demonstration that takes a source file and labels the sentences of its class comments. It is available as a Streamlit web application (demo/app.py) and as a command-line tool (demo/cli.py), both built on demo/inference.py. The pipeline has four steps:

1. **Extraction** (src/preprocess_comments.py): Java Javadoc comments directly before a class, interface, enum, or record (annotations allowed in between); Python class docstrings parsed with the ast module; Pharo class comments in both Tonel (.class.st, including traits) and File Out format. Method comments and file headers are ignored, as in the dataset.
2. **Normalisation** exactly as in the dataset: lower-casing, keeping letters, digits, and a few symbols, replacing a dot inside a word with ^, and splitting at line

breaks and sentence-ending dots. Rebuilding known classes of the dataset reproduces its sentences character for character.

3. **Classification** with the final model and the median cross-validation thresholds, using the same context prompt as in training.
4. **Presentation** of labels and probabilities grouped by class, with a CSV export.

Figure 8 shows the web application on a Java sample file. By default, the demo only allows labels of the file's language and guarantees at least one label per sentence. The pre-processing view (b) displays each original sentence next to the exact model input; a further tab shows the probability of every label for every sentence.

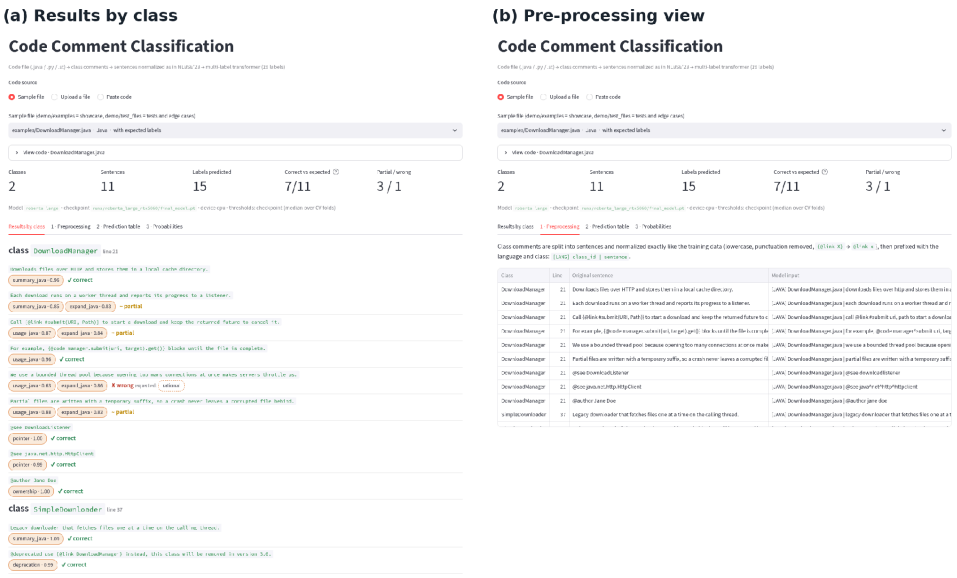


Figure 8: Web demo on DownloadManager.java: (a) predicted labels and probabilities grouped by class and checked against expected labels; (b) original sentences and model inputs

We prepared 8 showcase files (demo/examples, 86 annotated sentences covering all 19 labels) and 10 test files with rarer styles such as HTML lists, NumPy- and reST-style docstrings, nested classes and Pharo traits. With the main model, 54 of 86 showcase sentences (63%) and 53 of 94 test sentences (56%) receive exactly the expected label set; typical misses are Pharo section headers and Python "TODO:" notes. Because the expected labels were written by us following the dataset's conventions, these numbers are illustrative rather than a formal evaluation.

7. Limitations and Threats to Validity

7.1 Threshold tuning and the evaluation protocol

Thresholds are tuned on the validation folds on which the metrics are computed, so absolute scores are slightly optimistic; since all methods except the fixed-threshold RoBERTa baseline share this, comparisons remain fair. Our protocol also differs from the official NLBSE'23 protocol (19 binary tasks on a fixed test split), so our numbers are not directly comparable with published results; F1 (macro) is the closest of our metrics.

7.2 Sentence-level folds and the class name in the prompt

The folds are built at the sentence level. On the OOF predictions, 92.6% of the validation sentences belong to a class that also has sentences in the corresponding training fold. Since the context prompt contains the class name, and 80% of adjacent sentences in a class share their labels, the model can partly infer the labels of a sentence from the labels of other sentences of its class seen during training.

To estimate how strong this signal is, we evaluated a simple rule that never reads the sentence: for every validation sentence, it predicts the most frequent label(s) among the training sentences of the same class, and nothing for unseen classes. On the same five folds, this rule already reaches about **0.58 F1 (samples)**, as much as the RoBERTa baseline. In line with this, the exact match on the new demonstration files (56 to 63%) is lower than the 68% of cross-validation. Part of the +0.18 training gain of Section 5.3 may therefore come from the class name rather than from understanding the sentence, and may not transfer to classes that were never seen. A class-level split (GroupKFold on the class identifier) and an ablation of the prompt without the class name are needed to measure this precisely.

7.3 Statistical power and scope of the ablations

Five folds and one seed give limited statistical power: the +0.24 of the recipe is unambiguous, but small differences such as ModernBERT-large ($p = 0.03$) or pooling ($p = 0.36$) are weak evidence either way. The prompt, the loss, and the sampler were changed together, so their individual contributions are unknown.

CONCLUSION AND FUTURE WORK

This report addressed multi-label classification of class comment sentences on the NLBSE'23 dataset, following the four requirements of the assignment: ML and DL classifiers, suitable metrics, 5-fold cross-validation and a comparison with a RoBERTa baseline. Our single multi-label classifier on RoBERTa-large with LoRA, trained with a recipe for imbalanced labels, leads to the following findings:

- The recipe lifts the same RoBERTa-large from 0.5721 to **0.8124 F1 (samples)**, +0.240 over plain fine-tuning and +0.211 over TF-IDF + LR, on all five folds and on every metric.
- About +0.06 of the gain comes from the decision rule and about +0.18 from the training components; the threshold-independent PR-AUC confirms that the model's scores themselves improve.
- Four encoders under the same recipe end within 0.01 of each other; UniXcoder-base matches RoBERTa-large at about a third of the cost, and an ensemble of the four reaches 0.8382.
- The pooling strategy does not matter: masked mean and first-token pooling differ by 0.0035 F1 ($p = 0.36$).
- The remaining errors lie between semantically overlapping categories, mostly in Python, and the evaluation is likely optimistic because of sentence-level folds combined with the class name in the prompt.

Future work follows directly from the limitations: (1) class-level cross-validation to measure performance on unseen classes; (2) separate ablations of the prompt (with and without the class name), the loss and the sampler; (3) an evaluation on the official NLBSE'23 split with thresholds tuned on held-out training data, for comparison with published systems; and (4) distilling the four-model ensemble into a single model, ideally on the cheaper UniXcoder backbone.

REFERENCES

[1] P. Rani, S. Panichella, M. Leuenberger, A. Di Sorbo and O. Nierstrasz, "How to identify class comment types? A multi-language approach for class comment classification," *Journal of Systems and Software*, vol. 181, p. 111047, 2021.

[2] R. Kallis, G. Colavito, A. Al-Kaswan, L. Pascarella, O. Chaparro and P. Rani, "The NLBSE'23 tool competition," in *Proc. 2nd Int. Workshop on Natural Language-based Software Engineering (NLBSE)*, 2023.

[3] A. Al-Kaswan, M. Izadi and A. van Deursen, "STACC: Code comment classification using SentenceTransformers," in *Proc. 2nd Int. Workshop on Natural Language-based Software Engineering (NLBSE)*, 2023, arXiv:2302.13149.

[4] L. Tunstall, N. Reimers, U. E. S. Jo, L. Bates, D. Korat, M. Wasserblat and O. Pereg, "Efficient few-shot learning without prompts," arXiv:2209.11055, 2022.

[5] M.-L. Zhang and Z.-H. Zhou, "A review on multi-label learning algorithms," *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 8, pp. 1819–1837, 2014.

[6] K. Sechidis, G. Tsoumakas and I. Vlahavas, "On the stratification of multi-label data," in *Proc. ECML PKDD*, 2011, pp. 145–158.

[7] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional Transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[8] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[9] P. He, J. Gao and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," in *Proc. ICLR*, 2023.

[10] B. Warner, A. Chaffin, B. Clavié, O. Weller, O. Hallström, S. Taghadouini et al., "Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference," arXiv:2412.13663, 2024.

[11] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proc. ACL*, 2022, pp. 7212–7225.

[12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[13] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul and B. Bossan, "PEFT: State-of-the-art parameter-efficient fine-tuning methods," GitHub repository, 2022. [Online]. Available: https://github.com/huggingface/peft

[14] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020, pp. 38–45.

[15] T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár, "Focal loss for dense object detection," in *Proc. ICCV*, 2017, pp. 2980–2988.

[16] T. Ridnik, E. Ben-Baruch, N. Zamir, A. Noy, I. Friedman, M. Protter and L. Zelnik-Manor, "Asymmetric loss for multi-label classification," in *Proc. ICCV*, 2021, pp. 82–91.

[17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[18] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.

[19] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.

[20] "CodeCommentClassification," GitHub repository (original code base of this project). [Online]. Available: https://github.com/dang-nh/CodeCommentClassification

[21] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[22] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.